INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA

KULLIYYAH OF ENGINEERING

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 4: MOTION-ACTIVATED RFID SYSTEM

SECTION 1

GROUP 11

INSTRUCTOR: DR. WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL ABIDIN

Date of Experiment: Wednesday, 25th March 2026

Date of Submission: Wednesday, 1st April 2026

NO.	NAME	MATRIC NO.
1.	NURUL ANIS BINTI MOHD KHAIRI	2419788
2.	WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN	2320102
3.	SITI NURKHALISAH BINTI NAHZAMRI	2328482

ABSTRACT

This experiment presents the development and evaluation of a multi-layered, interactive access control system using an Arduino microcontroller and Python-based data processing. The study was conducted in three progressive phases: real-time motion detection, RFID credential validation, and the final integration of a Two-Factor Authentication (2FA) protocol.

In the initial phase, an MPU6050 accelerometer was utilized to detect circular "swirl" gestures through statistical analysis, specifically mean subtraction and variance thresholding of a 40-point data buffer. The second phase focused on digital security, employing a USB RFID scanner to authenticate a 10-digit Unique ID (UID) against a software-defined authorized list. The final phase synthesized these components, requiring both a valid RFID scan and a verified circular gesture—calculated via trigonometric "unwrapping" of rotational angles—to actuate a servo motor lock and provide LED feedback.

Results demonstrated that offloading complex mathematical analysis to Python significantly enhances the responsiveness and security of low-level hardware. The system successfully achieved a stable, bidirectional serial communication loop, proving that high-level software logic can effectively bridge the gap between raw sensor data and physical hardware actuation. This experiment validates the feasibility of using serial-linked architectures for robust, gesture-based security applications.

TABLE OF CONTENTS

INTRODUCTION 5
TASK 1: Interactive Motion Detection with MPU6050 6
1.0 MATERIALS AND EQUIPMENTS 6
1.1 EXPERIMENTAL SETUP 6
1.2 METHODOLOGY 6
- **a. Circuit Assembly** 6
- **b. Programming and Uploading** 7
- **c. Programming Logic** 7
- **d. Coding** 7

1.4 DATA COLLECTION 11
1.5 DATA ANALYSIS 11
1.6 RESULT 11
1.7 DISCUSSION 12
1.8 CONCLUSION 12
TASK 2: Introduction to RFID Integration 14
2.0 MATERIALS AND EQUIPMENTS 14
2.2 METHODOLOGY 14
- **a. Circuit Assembly** 14
- **b. Programming and Uploading** 15
- **c. Programming Logic** 15
- **d. Coding** 15

2.3 DATA COLLECTION 17
2.4 DATA ANALYSIS 17
2.5 RESULT 18
2.6 DISCUSSIONS 18
TASK 3: RFID and Servo Access System 20
3.0 MATERIALS AND EQUIPMENTS 20
3.2 EXPERIMENTAL SETUP 20
3.3 METHODOLOGY 20
- **a. Circuit Assembly** 20
- **b. Programming and Uploading** 21
- **c. Programming Logic** 21
- **d. Coding** 21

3.4 DATA COLLECTION 26
3.5 DATA ANALYSIS 26
3.6 RESULT 27

3.7 DISCUSSIONS 27
3.8 CONCLUSIONS 28
4.0 RECOMMENDATION 28
5.0 REFERENCES 29
6.0 APPENDICES 30
7.0 ACKNOWLEDGEMENT 31
8.0 STUDENTS DECLARATION 31
Certificate of Originality and Authenticity 31

INTRODUCTION

This experiment explores the integration between Arduino microcontrollers and the Python programming language to build an interactive, real-time control system. Generally, the project focuses on the capabilities of Serial Communication in linking physical sensors, such as the MPU6050 and RFID readers, with complex data processing logic on a computer. Through this combination, the developed system moves beyond simple digital inputs to perform statistical and trigonometric analyses, allowing for the dynamic control of output devices like LEDs and servo motors based on specific motion patterns and user identities.

In Task 1, the primary focus is on interactive motion detection using the MPU6050 inertial sensor. The goal of this section is to establish a bidirectional serial connection where accelerometer data is constantly streamed to Python for analysis using a 40-point data buffer. By utilizing mean and variance calculations, the system is designed to identify specific "swirl" motions while filtering out background noise, subsequently sending commands back to the Arduino to toggle an LED as visual feedback for the detected gesture.

Next, Task 2 introduces the use of Radio Frequency Identification (RFID) technology as an access control mechanism. The objective of this section is to interface a USB RFID scanner with a Python script to read the Unique ID (UID) of scanned cards. The data analysis involves comparing the detected UID against a pre-defined authorized list. If the card identity is verified, Python transmits a signal through the serial port to the Arduino to activate an LED for three seconds, demonstrating the fundamental concepts of credential-based digital security.

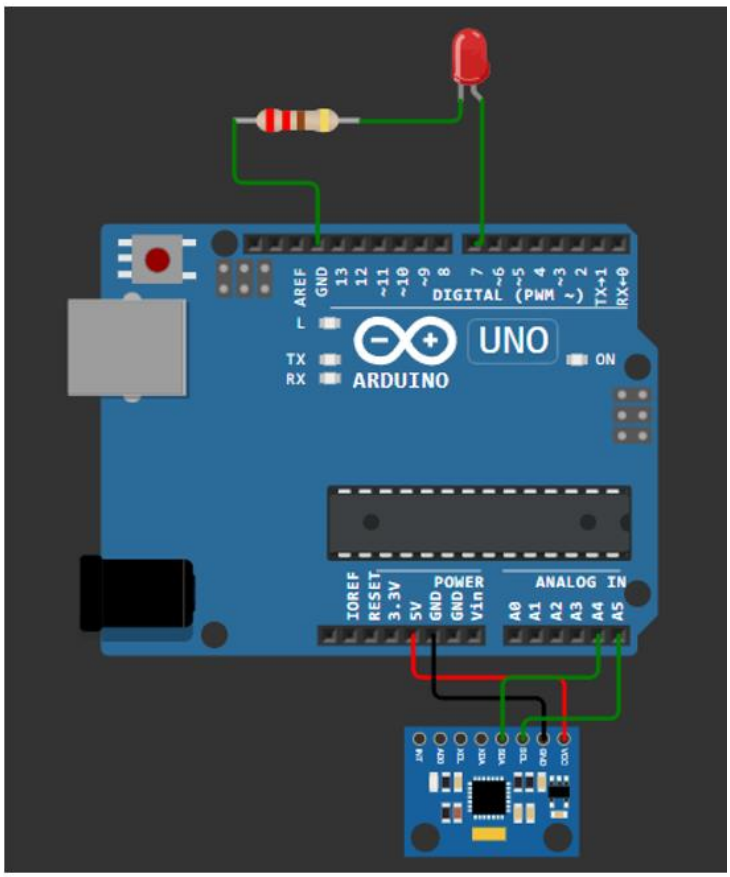
Finally, Task 3 synthesizes all previous components to build a comprehensive Two-Factor Authentication (2FA) access control system. In this phase, Python acts as the master controller managing a layered security logic; users must scan a valid RFID card and then perform a verified circular motion under the MPU6050 within a specific timeframe. This system not only tests the stability of inter-device communication but also proves the effectiveness of using a servo motor as a mechanical lock that only disarms when both verification factors—identity and gesture—are accurately met.

TASK 1: Interactive Motion Detection with MPU6050

1.0 MATERIALS AND EQUIPMENTS

- Arduino board + USB cable
- MPU6050 sensor
- Jumper wires/breadboard
- 1x LED (Any color)
- 1x 220 Ω Resistor
- Computer with Arduino IDE and Python installed

1.1 EXPERIMENTAL SETUP



1.2 METHODOLOGY

a. Circuit Assembly

- The VCC of MPU6050 is connected to pin 5V of Cytron .
- The GND pin of MPU6050 is connected to GND pin Cytron

- The SDA pin of MPU 6050 is connected to A4 pin of Cytron.
- The SCL pin of MPU 6050 is connected to A5 pin of Cytron
- Connect the LED to pin D7 and the 220 ohm resistor to GND pin of Cytron.

b. Programming and Uploading

- The Arduino IDE software was opened on the computer.
  - The correct board type (Arduino Uno) and communication port (COM port 9) were selected in the Tools menu.
  - The coding was uploaded and compiled with no syntax errors.
- The latest version of Python and PyCharm were downloaded on the computer.
  - During installation, ensure to check the box that says, "Add Python to PATH".
  - Install the required libraries by typing this single command and pressing Enter:
    pip install pyserial matplotlib
  - The Python script was uploaded and executed. To stop the program safely, click the red "Stop" square in PyCharm or press CTRL+C in the terminal window.

c. Programming Logic

- For the Arduino IDE the LED pin is initialized as the output.
- There is a if function being used, if the command receives 'O' the LED will light up.
- If the command receives 'F' the LED will turn off.
- There will be small delay for small stability.

d. Coding

- Arduino IDE

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

// --- DEFINE YOUR LED PIN HERE ---
const int ledPin = 7; // Fill in

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Start with LED off
}

void loop() {
  // 1. Read motion data
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  // 2. Send formatted data to Python (ax,ay,az)
  Serial.print(ax); Serial.print(',');
  Serial.print(ay); Serial.print(',');
  Serial.println(az);

  // 3. Listen for commands from Python to control the LED
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == 'O') {
      digitalWrite(ledPin, HIGH); // Turn LED ON
    }
    else if (cmd == 'F') {
      digitalWrite(ledPin, LOW);  // Turn LED OFF
    }
  }

  delay(50); // Small delay for stability
}
```

- PyCharm:

```python
import serial
import time
from collections import deque
import math

SERIAL_PORT = 'COM9'
BAUD_RATE = 9600

# Buffers to store the last 40 data points (approx. 2 seconds of motion)
history_x = deque(maxlen=40)
history_y = deque(maxlen=40)


def is_circular_motion(x_data, y_data):  1 usage
    """
    Analyzes an array of X and Y accelerometer data.
    Returns True if the data represents a circular motion, False otherwise.
    """
    if len(x_data) < 10:
        return False

    # --- STUDENTS: ADD YOUR CIRCLE DETECTION MATH HERE ---

    # Step 1: Find the center (mean) of the X and Y movement.
    mean_x = sum(x_data) / len(x_data)
    mean_y = sum(y_data) / len(x_data)

    x_centered = [x - mean_x for x in x_data]
    y_centered = [y - mean_y for y in y_data]

    # Step 2: Check Amplitude. Calculate the variance to ensure there is
    #         enough overall movement (ignore small jitters).
    var_x = sum((x ** 2 for x in x_centered)) / len(x_data)
    var_y = sum((y ** 2 for y in y_centered)) / len(x_data)

    amplitude = math.sqrt(var_x + var_y)
    if amplitude < 150:  # Adjust this
        return False
```

```python
    # Step 3: Check Symmetry. Check the ratio of X standard deviation to Y
    #         standard deviation to ensure it is round, not a straight line.
    std_x = math.sqrt(var_x)
    std_y = math.sqrt(var_y)

    if std_y == 0:
        return False

    ratio = std_x / std_y
    if ratio < 0.5 or ratio > 2.0:
        return False

    # Step 4: Check Continuous Rotation. Use the 2D Cross Product (x1*y2 - y1*x2)
    #         between consecutive centered points to see which way the path is turning.
    clockwise = 0
    counter_clockwise = 0

    for i in range(len(x_data) - 1):
        x1 = x_centered[i]
        y1 = y_centered[i]
        x2 = x_centered[i + 1]
        y2 = y_centered[i + 1]

        cross = x1 * y2 - y1 * x2

        if cross < 0:
            clockwise += 1
        elif cross > 0:
            counter_clockwise += 1
```

```python
    total = clockwise + counter_clockwise
    if total == 0:
        return False

    # Step 5: Tally the clockwise vs. counter-clockwise turns. If the movement
    #         is consistently turning in one direction (e.g., > 70%), return True!
    dominant = max(clockwise, counter_clockwise) / total
```

```python
    if dominant > 0.7:  # Adjust this
        return True

    return False # Change this to return True when your math detects a circle


def main():  1 usage
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.1)
    time.sleep(2)  # Wait for Arduino to reset
    print("System ready. Move the MPU6050 in a circle to toggle the LED...")

    # Keep track of the LED state
    led_is_on = False

    try:
        while True:
            raw = ser.readline().decode(errors='ignore').strip()
            if raw:
                parts = raw.split(',')

                if len(parts) >= 3:
                    try:
                        ax = int(parts[0])
                        ay = int(parts[1])

                        # Add new data to our buffers
                        history_x.append(ax)
                        history_y.append(ay)

                        # Only analyze if our buffer is full (40 points)
                        if len(history_x) == 40:
                            if is_circular_motion(history_x, history_y):
                                # Toggle the state
                                led_is_on = not led_is_on

                                if led_is_on:
                                    print("Circle Detected! Toggling LED: ON")
                                    ser.write(b'O')
```

```python
                                else:
                                    print("Circle Detected! Toggling LED: OFF")
                                    ser.write(b'F')

                                # DEBOUNCE: Clear the buffers!
                                # Forces the system to wait for 2 full seconds of
                                # NEW data before it can trigger another toggle.
                                history_x.clear()
                                history_y.clear()

                    except ValueError:
                        pass  # Ignore corrupted serial lines

    except KeyboardInterrupt:
        print("\nProgram terminated by user.")
    finally:
        ser.write(b'F')  # Ensure LED turns off when script stops
        ser.close()
        print("Serial connection closed.")


if __name__ == "__main__":
    main()
```

1.4 DATA COLLECTION

MOVEMENT	LED
Swirl the hand under MPU 6050	ON
Give F command	OFF

1.5 DATA ANALYSIS

The MPU6050 detects the physical force of hand swirly motion, and these values are sent as a string through the serial port. Once the phyton receives the data, it doesn't just look at one point, but it looks at a buffer of 40 points. By using the analysis in circular motion, the mean X and Y will be calculated, and they will be subtracted from every point. Other than that, it will calculate the variance, and it will ignore the movement that is too small. For example, if the variance is below 150.

Next, the concept of the Command 'O' is that if the state becomes True, the phyton will send the byte 'b0' to the Arduino. The Arduino will read the '0' entered if (cmd=='0') block and execute digitalWrite(ledPin, HIGH), physically lighting the LED. The phyton sends b'F' to the serial port. The character will be matched by the Arduino in else if (cmd=='F') and executes digitalWrite(ledPin, LOW).

1.6 RESULT

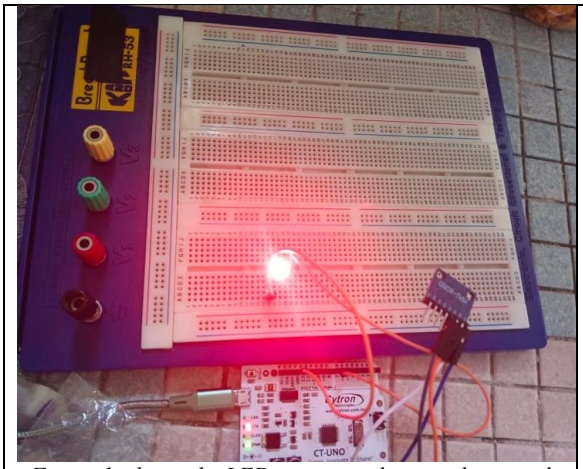


Figure 1 shows the LED turns on when swirls around

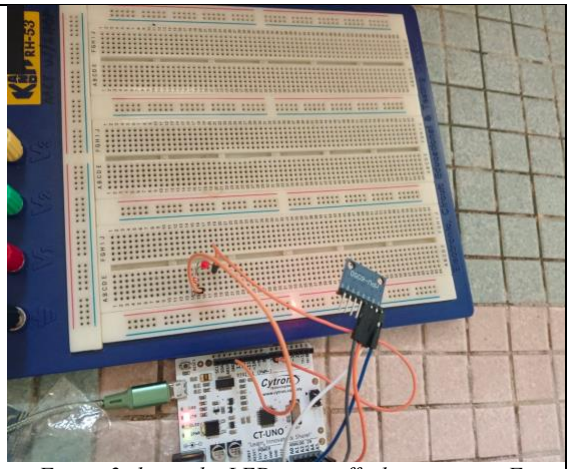


Figure 2 shows the LED turns off when entering F command

1.7 DISCUSSION

The integration of the MPU6050 with Python demonstrated the necessity of motion recognition. Rather than relying on instantaneous acceleration values, which are prone to noise and gravitational bias, the system utilised a 40-point buffer to analyse patterns over time. By calculating the mean of the X and Y axes and subtracting it from each data point, the software effectively isolates the dynamic circular swirl motion from static environmental factors. Furthermore, implementing a variance threshold of 150 served as a critical noise filter, which ensured that the system was only reacting to hand movements. This prevents minor sensor jitter or hand tremors from triggering false positives in the LED state.

The success of the experiment relied on a bidirectional serial communication loop that bridged high-level data processing with physical hardware control. Once the Python script identified a valid circular motion, it transmitted specific byte commands—b'O' for "On" and b'F' for "Off"—back to the Arduino. This approach highlighted the efficiency of character-based protocols in embedded systems, as the Arduino's if (cmd == 'O') logic allowed for near-instantaneous execution of the digitalWrite command. While effective, the system's reliability is inherently tied to the fixed buffer size and hard-coded thresholds, suggesting that while the closed-loop communication is robust, the detection sensitivity would need to be calibrated for different user movement speeds or sensor orientations.

1.8 CONCLUSION

In conclusion, Task 1 of the experiment is successful in establishing a stable, bidirectional communication link between the Arduino and Python for real-time motion analysis. By utilising the MPU6050 accelerometer and a 40-point data buffer, the system effectively distinguished deliberate circular gestures from static noise and minor vibrations through mean subtraction and variance thresholding. The successful toggling of the physical LED based on the Python-processed "circle" detection confirms that the serial protocol was

robust and that the latency between data acquisition, analysis, and command execution was minimal enough for interactive use.

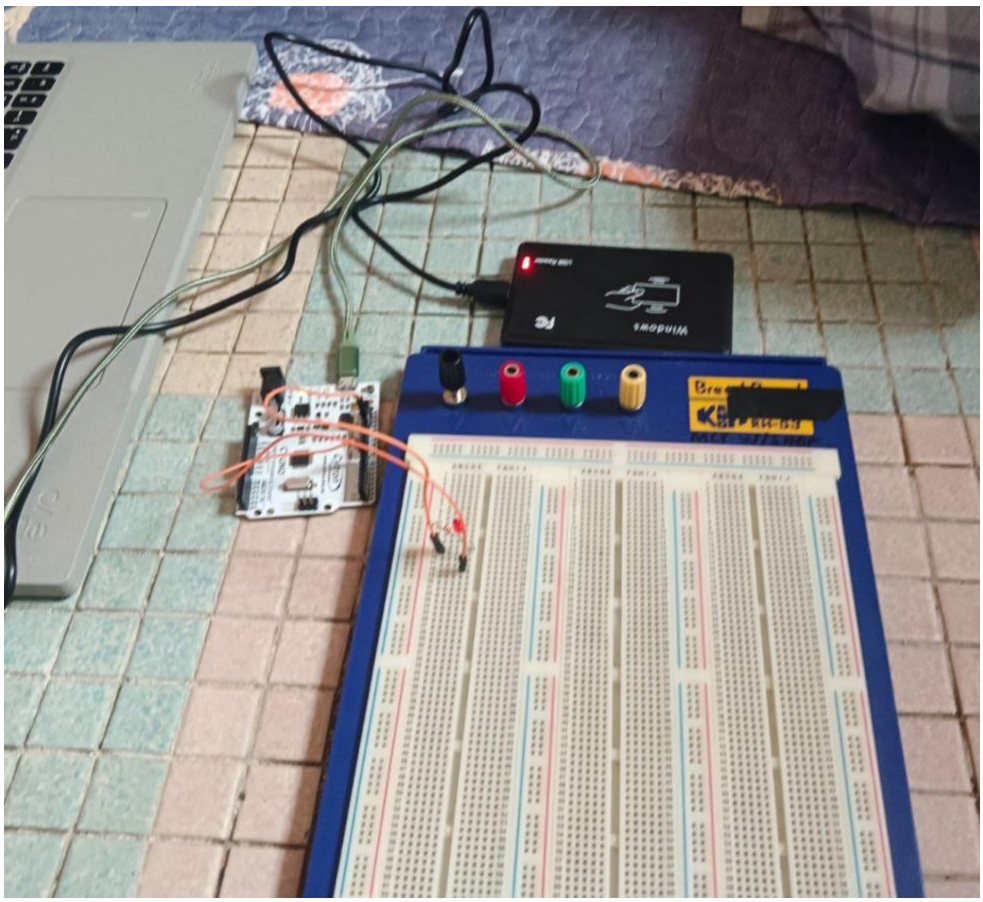
This success validates the project's foundational logic: that raw sensor data can be transformed into actionable commands through a combination of statistical filtering and structured serial messaging. While the detection parameters (such as the variance threshold of 150) were effective for this specific setup, the experiment proved that the core framework is a viable method for developing more complex gesture-controlled hardware interfaces.

TASK 2: Introduction to RFID Integration

2.0 MATERIALS AND EQUIPMENTS

- Arduino board + USB cable
- USB RFID Scanner + RFID tags/cards
- 1x LED (Any color)
- 1x 220 Ω Resistor
- Jumper wires and breadboard

2.1 EXPERIMENTAL SETUP



2.2 METHODOLOGY

a. Circuit Assembly

- The USB RFID scanner is being connected to the computer's USB port.
- The Anode of the LED is connected to a digital pin of the Cytron
- The Cathode of the LED is connected to 220-ohm resistor and the other end of the resistor is connected to Cytron's GND.

b. <u>Programming and Uploading</u>

- The Arduino IDE software was opened on the computer.
  - The correct board type (Arduino Uno) and communication port (COM port 7) were selected in the Tools menu.
  - The coding was uploaded and compiled with no syntax errors.

- The latest version of Python and PyCharm were downloaded on the computer.
  - During installation, ensure to check the box that says, "Add Python to PATH".
  - Install the required libraries by typing this single command and pressing Enter:
    pip install pyserial matplotlib
  - The Python script was uploaded and executed. To stop the program safely, click the red "Stop" square in PyCharm or press CTRL+C in the terminal window.
  - Ensure that Arduino IDE is not opened when running Python code in PyCharm.

c. <u>Programming Logic</u>

- There is 'If' function being used, if the card is being tapped on the rfid Reader, it will give the command to the Arduino.
- if the command receive '1' The led will light up but if the command receive '0' the led will turn off.
- If there is no card being tapped on the RFID, there is no command from phycharm to Arduino and the LED will turn off

d. <u>Coding</u>

- Arduino IDE:

```
const int ledPin = 7; // Fill in

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Start with LED OFF
}

void loop() {
  // Listen for commands from Python
  if (Serial.available() > 0) {
    char cmd = Serial.read();

    if (cmd == '1') {
      digitalWrite(ledPin, HIGH); // Turn LED ON
    }
    else if (cmd == '0') {
      digitalWrite(ledPin, LOW);  // Turn LED OFF
    }
  }
}
```

- PyCharm:

```python
import serial
import time

SERIAL_PORT = 'COM9'
BAUD_RATE = 9600
AUTHORIZED_CARD = ("0013078065")

def main():  1 usage
    # Establish connection with the Arduino
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)  # Wait for Arduino to reset

    print("RFID System Ready.")
    print("Scan a card to test the system. Press CTRL+C to stop.")

    try:
        while True:
            uid = input("\nTap card: ").strip()

            if uid == AUTHORIZED_CARD:
                print(f"[{uid}] - Card Authorized! Turning LED ON...")
                arduino.write(b'1')  # Send ON command to Arduino

                time.sleep(3)  # Keep LED on for 3 seconds

                arduino.write(b'0')  # Send OFF command
                print("LED OFF. Ready for next scan.")

            else:
                print(f"[{uid}] - Unauthorized Card! LED remains OFF.")

    except KeyboardInterrupt:
        print("\nExiting program.")
    finally:
        # Ensure the LED is safely turned off when the program closes
        arduino.write(b'0')
        arduino.close()
        print("Serial connection closed.")

if __name__ == "__main__":
    main()
```

2.3 DATA COLLECTION

Movement	LED
Tapping the card on RFID reader	ON
Not tapping the card on RFID reader	OFF

2.4 DATA ANALYSIS

The data collection process for this section focused on the successful capture and validation of the 10-digit Unique ID (UID) transmitted by the USB RFID scanner. Unlike the MPU6050, which streamed continuous raw data, the RFID scanner acted as a Human Interface Device (HID), automatically inputting the specific string ‘0013078065’ followed by a carriage return upon a successful tap. The Python script’s input().strip() function served as the primary data capture mechanism, isolating the numerical UID from any trailing whitespace or newline characters. This ensured that the comparison against the AUTHORIZED_CARD constant was based on an exact string match, providing a binary outcome for the access control logic.

The analysis of this data was governed by a conditional logic gate that determined the hardware's physical state. When the scanned UID matched the authorized credentials, Python

initiated a specific timing sequence, transmitting the byte b'1' to the Arduino via the ‘COM9’ serial port. This triggered the digitalWrite(ledPin, HIGH) function within the Arduino’s loop. A crucial part of the data handling was the inclusion of a time.sleep(3) delay in the Python script; this created a controlled state duration where the LED remained active for exactly three seconds before the script sent the b'0' termination command. In instances where an unauthorized card was scanned, the mismatch in the string comparison effectively blocked the serial transmission, maintaining the LED in a LOW state and demonstrating a functional software-based security filter.

2.5 RESULT

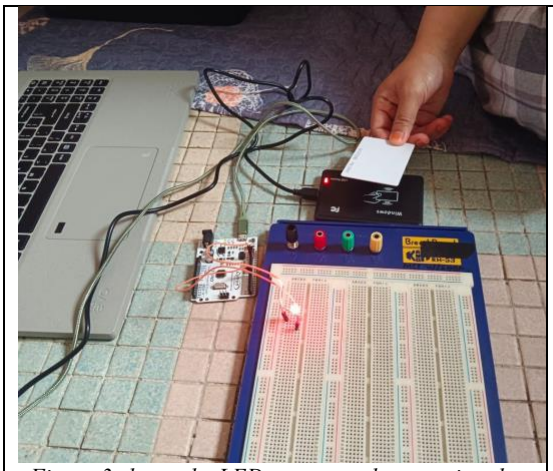


Figure 3 shows the LED turns on when tapping the card

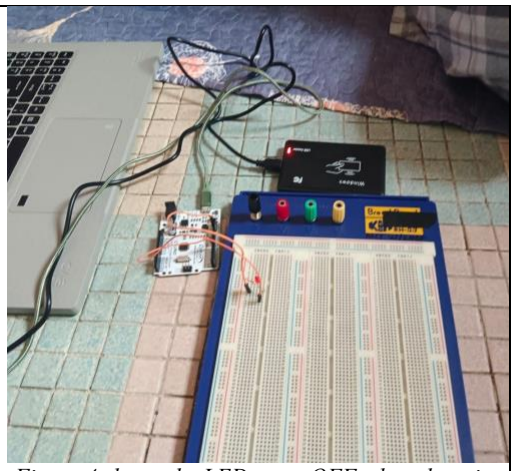


Figure 4 shows the LED turns OFF when there is no cards

2.6 DISCUSSIONS

The implementation of the RFID system demonstrated the effectiveness of using a high-level language like Python as a "security gatekeeper" for low-level hardware. By offloading the authentication logic to the Python script, the system could easily manage an authorized database (the AUTHORIZED_CARD variable) without requiring the Arduino’s memory to store complex strings. This setup highlighted the advantage of Human Interface Device (HID) integration; since the USB scanner emulates a keyboard, the data acquisition was seamless, requiring only basic string manipulation via strip() to ensure the input matched the stored credentials. The three-second active window is created by the time.sleep(3) command showcased a simple but effective way to handle state-based control, where the software dictates the duration of a hardware event.

A key observation during this part of the lab was the reliability of the serial handshake between the PC and the Arduino. Because the Arduino was programmed as a "passive listener" (waiting for '1' or '0'), the system remained highly responsive. However, the use of time.sleep() in Python creates a "blocking" effect, meaning the script cannot process a new card scan until the 3-second LED cycle is complete. In a real-world security application, this could be improved using non-blocking timing methods to allow for continuous scanning.

Overall, this part of the experiment successfully illustrated the core principles of Access Control Systems, proving that software-defined logic can effectively secure physical environments by mediating between input sensors and output actuators.

2.7 CONCLUSIONS

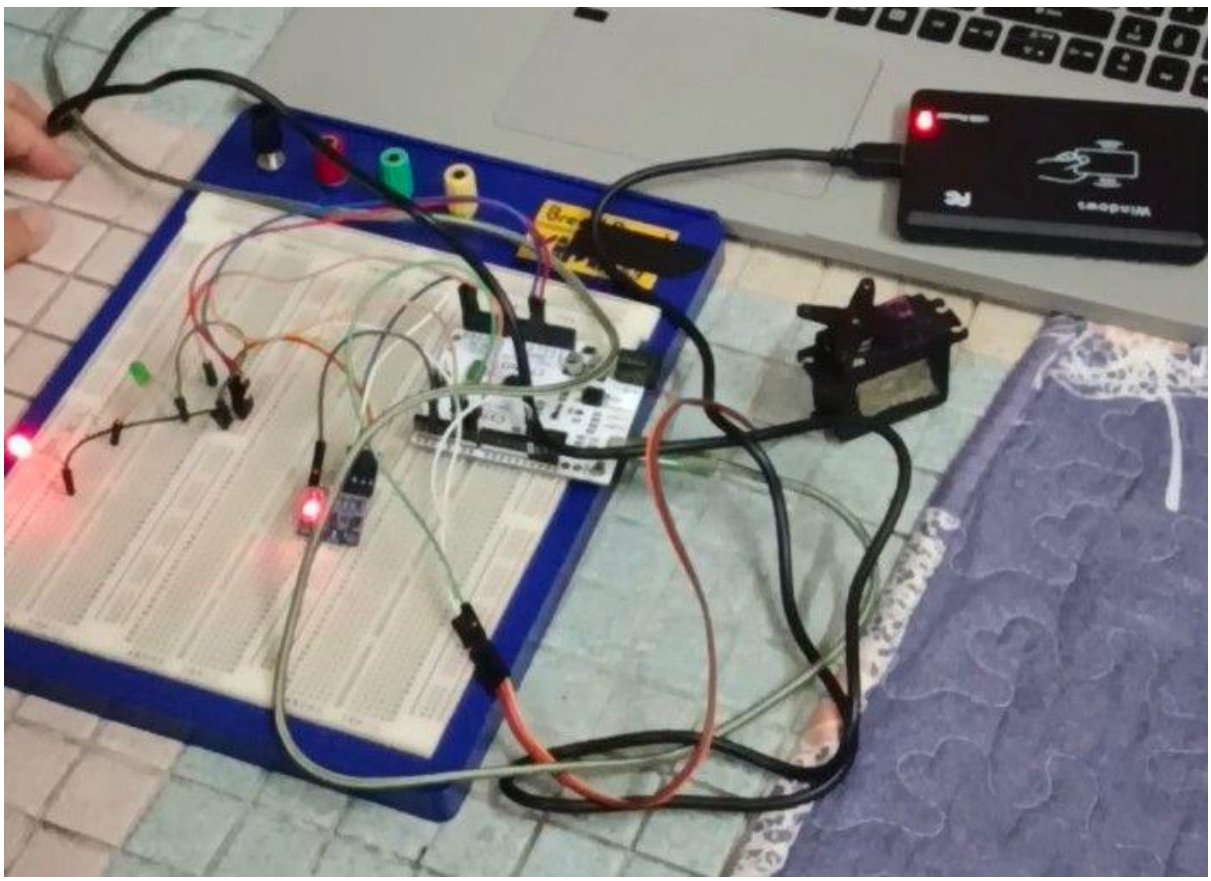
In conclusion, Part 2 of the lab was successful in demonstrating a functional RFID access control system through the integration of a USB scanner and Python-mediated serial communication. By correctly identifying the specific UID 0013078065, the software proved capable of distinguishing authorized credentials from unauthorized inputs and executing a timed hardware response on the Arduino. This exercise successfully validated the concept of using high-level software logic to manage physical security protocols, confirming that serial commands can effectively bridge the gap between digital authentication and real-world hardware actuation.

TASK 3: RFID and Servo Access System

3.0 MATERIALS AND EQUIPMENTS

- Arduino board + USB cable
- MPU6050 sensor
- USB RFID Scanner + RFID tags/cards
- 1x Servo Motor (SG90)
- 1x Green LED
- 1x Red LED
- 2x 220 Ω Resistor
- Jumper wires and breadboard

3.2 EXPERIMENTAL SETUP



3.3 METHODOLOGY

a. Circuit Assembly

- Connect the signal of servo motor to pin D6 of Cytron,VCC to 5V and the GND of servo motor to GND pin of cytron.

- The Anode of the green LED is connected to the pin D5 and the Cathode to the GND through 220 ohm resistor.
- The Anode of the red LED is connected to pin D7 and the Cathode to GND through 220 ohm resistor.

b. Programming and Uploading

- The Arduino IDE software was opened on the computer.
  - The correct board type (Arduino Uno) and communication port (COM port 7) were selected in the Tools menu.
  - The coding was uploaded and compiled with no syntax errors.

- The latest version of Python and PyCharm were downloaded on the computer.
  - During installation, ensure to check the box that says, "Add Python to PATH".
  - Install the required libraries by typing this single command and pressing Enter:
    pip install pyserial matplotlib
  - The Python script was uploaded and executed. To stop the program safely, click the red "Stop" square in PyCharm or press CTRL+C in the terminal window.
  - Ensure that Arduino IDE is not opened when running Python code in PyCharm.

c. Programming Logic

- If the card is being tapped on the RFID reader for the second time, the If the command at the Arduino received 1 the green LED will light up and the red LED will turn off.
- If the command at the Arduino received '0', the red LED will light up and the green LED will turn off
- If the card is being tapped on RFID reader once, the green LED will light up and the servo motor will rotate
- red LED will turn on and the green LED will turn off. The servo motor also will stop rotating.

d. Coding

- Arduino IDE

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo gateServo;

const int greenLedPin = 5;
const int servoPin = 6;
const int redLedPin = 7;

const unsigned long streamDurationMs = 2000; // Stream data for 2 seconds
const unsigned long sampleDelayMs = 50;      // 20 Hz sample rate

void setup() {
  pinMode(greenLedPin, OUTPUT);
  pinMode(redLedPin, OUTPUT);

  gateServo.attach(servoPin);
  gateServo.write(0); // Start at closed position

  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  // Default state: red LED ON (idle/unauthorized)
  digitalWrite(redLedPin, HIGH);
  digitalWrite(greenLedPin, LOW);
}

void openGate() {
  for (int pos = 90; pos <= 180; pos += 2) {
    gateServo.write(pos);
    delay(15);
  }
  Serial.println("Servo opened (90 -> 180)");
}
```

```
44  void closeGate() {
45    for (int pos = 180; pos >= 90; pos -= 2) {
46      gateServo.write(pos);
47      delay(15);
48    }
49    Serial.println("Servo closed (180 -> 90)");
50  }
51
52  void wrongCardFlash() {
53    digitalWrite(redLedPin, HIGH);
54    digitalWrite(greenLedPin, LOW);
55    delay(100);
56    // Blink red LED 3 times
57    for (int i = 0; i < 3; i++) {
58      digitalWrite(redLedPin, HIGH);
59      delay(150);
60      digitalWrite(redLedPin, LOW);
61      delay(150);
62    }
63    digitalWrite(redLedPin, HIGH); // Return to idle red ON
64  }
65
66  void loop() {
67    if (Serial.available()) {
68      char cmd = Serial.read();
69
70      if (cmd == '1') {
71        // Access granted
72        digitalWrite(greenLedPin, HIGH);
73        digitalWrite(redLedPin, LOW);
74        openGate();
75      }
76      else if (cmd == '0') {
77        // System disarmed (Lock)
78        digitalWrite(greenLedPin, LOW);
79        digitalWrite(redLedPin, HIGH);
80        closeGate();
81      }
82      else if (cmd == 'X') {
83        // Wrong card feedback
84        wrongCardFlash();
85      }
86      else if (cmd == 'R') {
87        // Python requested motion data
88        unsigned long start = millis();
89        while (millis() - start < streamDurationMs) {
90          int16_t ax, ay, az, gx, gy, gz;
91          mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
92
93          Serial.print(ax); Serial.print(',');
94          Serial.print(ay); Serial.print(',');
95          Serial.print(az); Serial.print(',');
96          Serial.print(gx); Serial.print(',');
97          Serial.print(gy); Serial.print(',');
98          Serial.println(gz);
99
100         delay(sampleDelayMs);
101       }
102     }
103   }
104 }
```

- PyCharm:

```python
import serial
import time
import math

# --- CONFIGURATION ---
SERIAL_PORT = 'COM9' # Update with your Arduino port
BAUD_RATE = 9600
AUTHORIZED_ID = "0013078065"

# --------------------

def detect_circle(arduino):  1 usage
    """
    Asks Arduino to stream a burst of MPU data, calculates the rotation angles,
    and decides whether a circular motion happened.
    """
    print("\n[!] Requesting samples from Arduino… Move in a circle NOW!")
    arduino.reset_input_buffer()
    arduino.write(b'R')  # Trigger Arduino data stream

    points = []
    start_time = time.time()
    timeout = 3.0

    # Read until timeout (Arduino stops streaming automatically after 2s)
    while time.time() - start_time < timeout:
        raw = arduino.readline().decode(errors='ignore').strip()
        if not raw:
            continue

        parts = raw.split(',')
        if len(parts) >= 6:
            try:
                ax = int(parts[0])
                ay = int(parts[1])
                az = int(parts[2])
                points.append((ax, ay, az))
            except ValueError:
                continue
```

```python
    print(f"Collected {len(points)} samples.")
    if len(points) < 8:
        print("Not enough motion data - try moving more continuously.")
        return False

    # Build angles from (ax, ay) and unwrap to follow continuous rotation
    angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]

    unwrapped = [angles[0]]
    for a in angles[1:]:
        prev = unwrapped[-1]
        diff = a - prev

        # Unwrap angles to avoid -180/180 discontinuity leaps
        if diff > 180:
            diff -= 360
        elif diff < -180:
            diff += 360
        unwrapped.append(prev + diff)

    # Compute net rotation and range
    net_rotation = unwrapped[-1] - unwrapped[0]
    rotation_range = max(unwrapped) - min(unwrapped)

    print(f"Net rotation: {net_rotation: .1f}°, Range: {rotation_range: .1f}°")

    # Lenient detection logic (either a decent net rotation OR wide range)
    if abs(net_rotation) > 100 or rotation_range > 140:
        print("-> Circle gesture detected!")
        return True

    # Extra check: count direction-consistent large angle deltas
    deltas = [unwrapped[i + 1] - unwrapped[i]
    for i in range(len(unwrapped) - 1)]
    large_moves = sum(1 for d in deltas if abs(d) > 10)
```

```python
    if large_moves >= max(6, len(points) // 4):
        print("→ Motion had multiple consistent movements - accepting as circle.")
        return True

    print("→ Motion insufficient.Try a clearer circular motion.")
    return False


def main():  1 usage
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)  # Wait for Arduino to reset

    print("System Ready.Waiting for RFID…")
    armed = False  # System starts locked

    try:
        while True:
            # USB RFID acts as a keyboard
            card = input("\nTap card: ").strip()

            if card == AUTHORIZED_ID:
                armed = not armed  # Toggle state

                if armed:
                    print("Card Accepted - Please perform circular motion verification!")

                    if detect_circle(arduino):
                        print("Gesture OK -> System UNLOCKED(LED Green)")
                        arduino.write(b'1')
                    else:
                        print("Wrong gesture -> Try again!")
                        armed = False  # Revert state if gesture fails
                else:
                    print("System Disarmed -> System LOCKED(LED Red)")
                    arduino.write(b'0')
```

```python
            else:
                print("WRONG CARD -> Access Denied")
                arduino.write(b'X')

    except KeyboardInterrupt:
        print("\nExiting.")
    finally:
        arduino.write(b'0')  # Ensure locked on exit
        arduino.close()

if __name__ == "__main__":
    main()
```

3.4 DATA COLLECTION

MOVEMENT	GREEN LED	RED LED	SERVO MOTOR
Tapping the card on RFID reader once and swirl the hand under MPU 6050	ON	OFF	ROTATE
Tapping the card on RFID reader second time	OFF	ON	NOT ROTATING

3.5 DATA ANALYSIS

The data analysis for the final task focused on the integration of two distinct data streams to form a "Two-Factor Authentication" (2FA) security protocol. The first layer of data was the categorical input from the USB RFID reader. As seen in the Python script, the system initialized in a LOCKED state (armed = False). Upon tapping the authorized card (0013078065), the script toggled the 'Armed' state, triggering a secondary data request. Unlike previous tasks, the software did not immediately grant access; instead, it called the detect_circle(arduino) function, which initiated a 3-second window for gathering rotational acceleration data from the MPU6050.

The second layer of analysis involved complex trigonometric processing of the raw accelerometer values (a_x, a_y, a_z). Python calculated the rotation angles using math.atan2(p[1], p[0]) function and applied an "unwrapping" algorithm to track continuous circular motion without discontinuity at the 180° boundary. Access was only granted if the data met a specific threshold—either a net rotation exceeding 100° or a total rotation range over 140°. When both the RFID UID matched and the gesture verification returned True, the Python script sent a b'1' command to the Arduino. This resulted in a state change where the Green LED was activated, the Red LED was deactivated, and the Servo motor rotated to the "Unlocked" position. A second tap of the card successfully toggled the system back to a 'Disarmed' state, sending a b'0' command to reset the hardware, demonstrating a fully integrated, logic-driven access control loop.

3.6 RESULT

```
C:\Users\acer\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\acer\PyCharmMiscProject\script5.py
System Ready.Waiting for RFID…

Tap card: 0013078065
Card Accepted - Please perform circular motion verification!

[!] Requesting samples from Arduino… Move in a circle NOW!
Collected 38 samples.
Net rotation: -305.5°, Range:  334.2°
-> Circle gesture detected!
Gesture OK -> System UNLOCKED(LED Green)

Tap card: 0013078065
System Disarmed -> System LOCKED(LED Red)
```

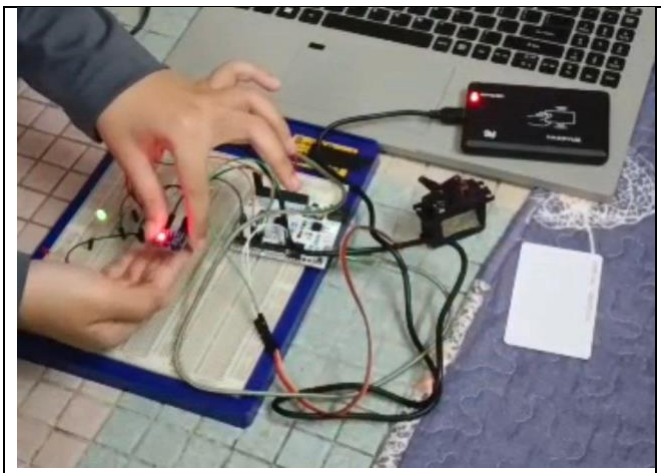


Figure 5 shows the green led turns on after tapping the card and swirl the hand under MPU 6050

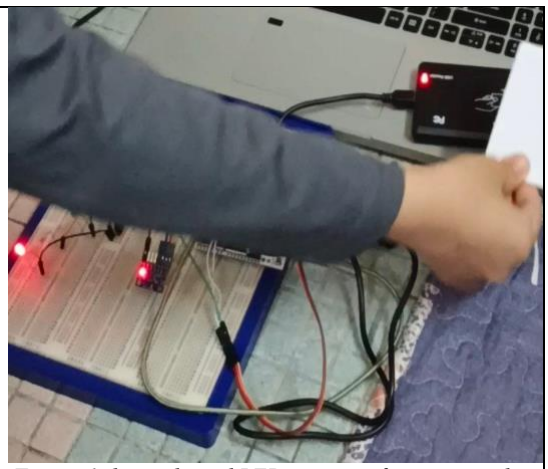


Figure 6 shows the red LED turn on after tapping the card for the second time

3.7 DISCUSSIONS

The final task successfully integrated all previous components into a sophisticated Two-Factor Authentication (2FA) system, where physical access (RFID) was paired with behavioural verification (MPU6050 gesture). This architecture significantly enhances security; even if an unauthorized user possessed the correct RFID card, they would fail the "gesture challenge" required to move the servo motor. The discussion of this system centers on the state-machine logic managed by Python. By using a toggle variable (armed = not armed), the script could distinguish between an initial entry attempt and a deliberate "lock-down" command. This demonstrates how software can maintain a "memory" of the system's current status, allowing the same physical input (a card tap) to result in two different hardware outputs depending on the context.

A major technical highlight in this part was the transition from simple variance checks to trigonometric motion tracking. By using math.atan2 and an "unwrapping" algorithm, the

system moved beyond detecting "shaking" to specifically tracking angular displacement. This allowed the Python script to calculate the net_rotation, ensuring that only a deliberate circular path would trigger the b'1' unlock command. The use of the servo motor as the final actuator provided a tangible "mechanical lock" simulation, proving that the serial communication protocol is robust enough to handle multiple data types—UID strings, accelerometer coordinates, and motor position commands—simultaneously. While the system was successful, the reliance on a 3-second timeout for the gesture implies that the user must be synchronized with the software's "Requesting samples" prompt, suggesting that future iterations could benefit from a more flexible, interrupt-driven detection window.

3.8 CONCLUSIONS

In conclusion, Task 3 successfully demonstrated the implementation of a Two-Factor Authentication (2FA) system by integrating RFID identification with MPU6050 gesture verification. The experiment proved that Python can effectively serve as a master controller, processing complex trigonometric data to "unwrap" rotational angles and determine if a user's physical motion meets specific security criteria. By successfully toggling the system between a "Locked" state (Red LED ON, Servo at rest) and an "Unlocked" state (Green LED ON, Servo rotated) based on both a valid UID and a verified circular gesture, the lab validated the robustness of bidirectional serial communication. This final task confirms that combining multiple sensor inputs with software-defined logic creates a significantly more secure and interactive access control interface than single-sensor systems.

4.0 RECOMMENDATION

For **Task 1**, one recommendation is to implement a dynamic scaling factor for the variance threshold instead of using a hard-coded value of 150. This is because different users may move at different speeds and intensities, which will affect the system. Thus, if motion detection is more refined, we can improve the reliability of the circular motion detection by constructing a "calibration phase" where the user performs a few baseline movements. Likewise, systems that are calibrated with short subject-specific procedures like simple postures or movement can reach a high detection accuracy (~90%) without exhaustive per-user motion data, showing that brief calibration can tune thresholds/features to an individual's signal level and movement style.

Furthermore, increasing the sampling rate of MPU6050 and the size of the data buffer would allow for a higher resolution of the motion path, which reduces the likelihood of "false negatives" during slower hand movements. For IMU-based orientation and high-speed cyclic motions, accuracy improves as sampling increases up to movement-specific plateaus (e.g., ~100 Hz for walking, ~200–400 Hz for faster cyclic actions). Similarly, a large buffer improves the robustness of motion characterization and phase extraction, particularly for subtle or slow displacements, which then allows better trajectory reconstruction and reduces missed detection.

For **Task 2**, one recommendation is to transition from a blocking input method to an asynchronous or non-blocking read approach in Python. In the current setup, the input() function halts the entire script until a card is scanned. By using a serial communication library such as pyserial to continuously poll the buffer in the background, the system can perform other tasks (for example, blinking a "Ready" LED) while waiting for a card. This improves both responsiveness and user experience. Additionally, implementing an "Unauthorized Attempt" counter that triggers a temporary lockout after three incorrect scans would enhance the overall security of the system. This mechanism helps prevent brute-force attempts and aligns the design more closely with common industry security practices.

For **Task 3**, the user experience can be improved by adding real-time visual or auditory feedback during the 3-second "Gesture Verification" window. For example, activating a buzzer or a flickering yellow LED when the Arduino is ready to receive MPU6050 data would clearly indicate to the user when to start performing the swirl motion. This feedback helps reduce timing uncertainty and increases the likelihood of successful gesture authentication. From a security perspective, the system can be further strengthened by using non-volatile memory (EEPROM) on the Arduino to store the "Armed" state. By preserving whether the system is Locked or Unlocked across power cycles, the system will not unintentionally revert to a disarmed or default state after a power interruption. This ensures more robust security and consistent behavior in real-world deployment scenarios.

5.0 REFERENCES

Fan, B., Zhang, L., Cai, S., Du, M., Liu, T., Li, Q., & Shull, P. (2025). Influence of Sampling Rate on Wearable IMU Orientation Estimation Accuracy for Human Movement Analysis. *Sensors*, *25*(7), 1976. https://doi.org/10.3390/s25071976

Instructables. (2016, December 5). *Arduino + MFRC522 RFID READER*. Instructables. https://www.instructables.com/Arduino-MFRC522-RFID-READER/

Last Minute Engineers. (2018, July 30). *What is RFID? How It Works? Interface RC522 RFID Module with Arduino*. Last Minute Engineers; Last Minute Engineers. https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/

Miyake, T., Yamamoto, S., Hosono, S., Funabashi, S., Cheng, Z., Zhang, C., Tamaki, E., & Sugano, S. (2021). Gait Phase Detection Based on Muscle Deformation with Static Standing-Based Calibration. *Sensors*, *21*(4), 1081. https://doi.org/10.3390/s21041081

Santos, R. (2016, March 23). *MFRC522 RFID Reader with Arduino Tutorial | Random Nerd Tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

Santos, S. (2018, May 24). *Arduino Time Attendance System with RFID | Random Nerd Tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid

6.0 APPENDICES

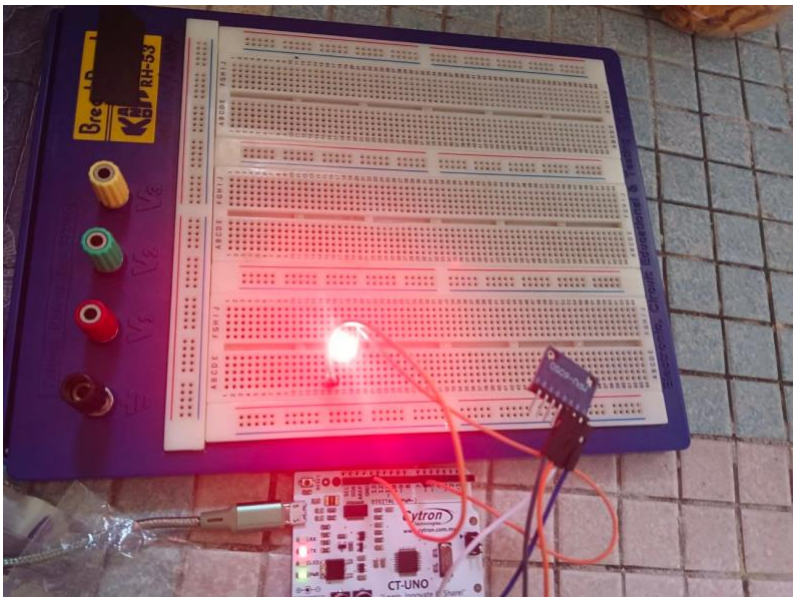


Figure 7 shows the circuit for task 1

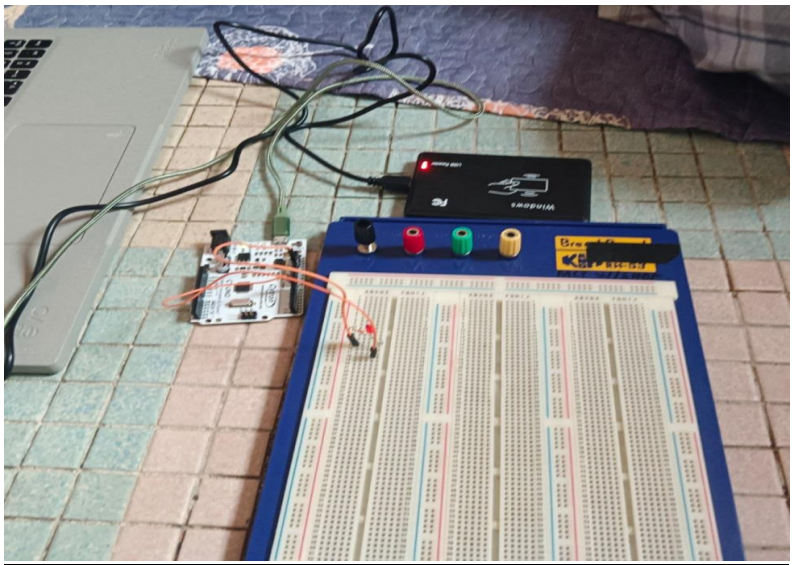


Figure 8 shows the circuit for task 2

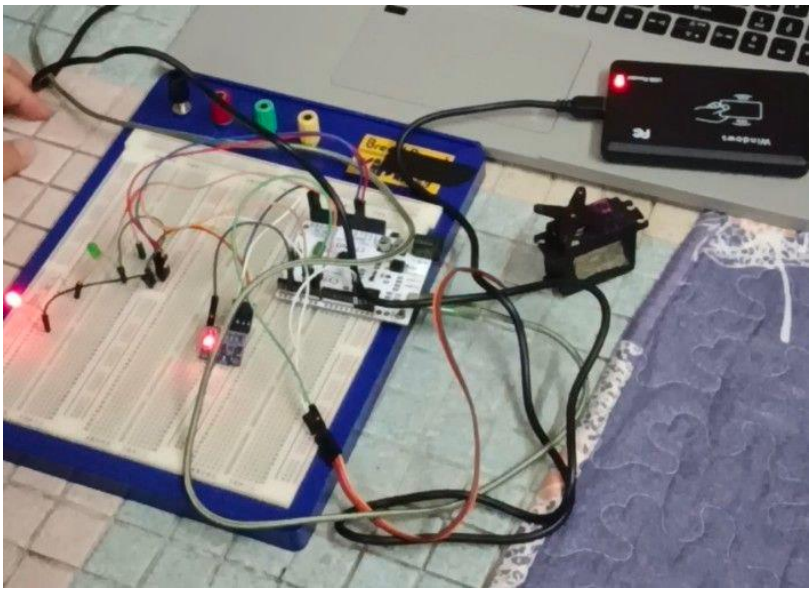


Figure 9 shows the circuit for task 3

7.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help, encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

8.0 STUDENTS DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behaviour. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein has not been done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed

to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributor to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final report has been verified by us.

Signature: *nurulanis*	Read [/]
Name: NURUL ANIS BINTI MOHD KHAIRI	Understand[/]
Matric Number: 2419788	Agree [/]
Signature: *athirahmeor*	Read [/]
Name: WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN	Understand[/]
Matric Number: 2320102	Agree [/]
Signature: *nurkhalisah*	Read [/]
Name: SITI NURKHALISAH BINTI NAHZAMRI	Understand[/]
Matric Number: 2328482	Agree [/]